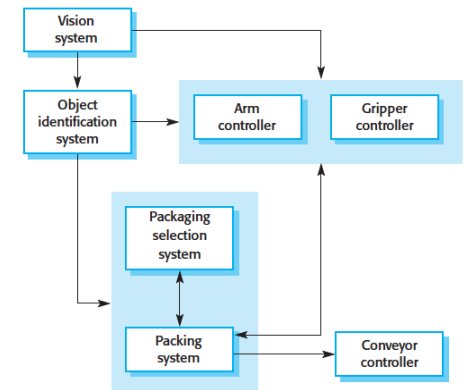


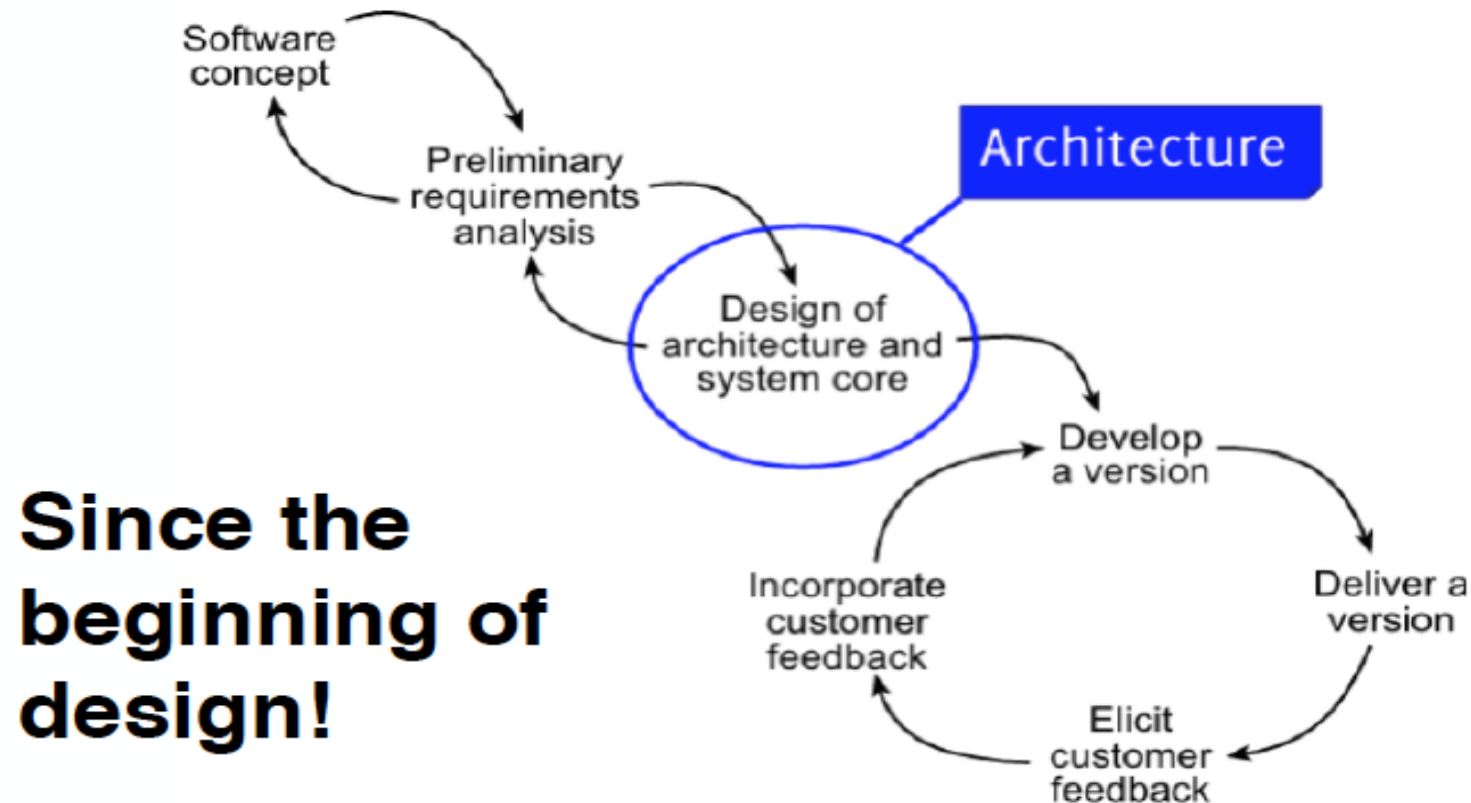
# COSC 612/AIT 624

## Architectural Design

Dr. Akshita Maradapu Vera Venkata Sai  
Department of Computer and Information Sciences  
Towson University

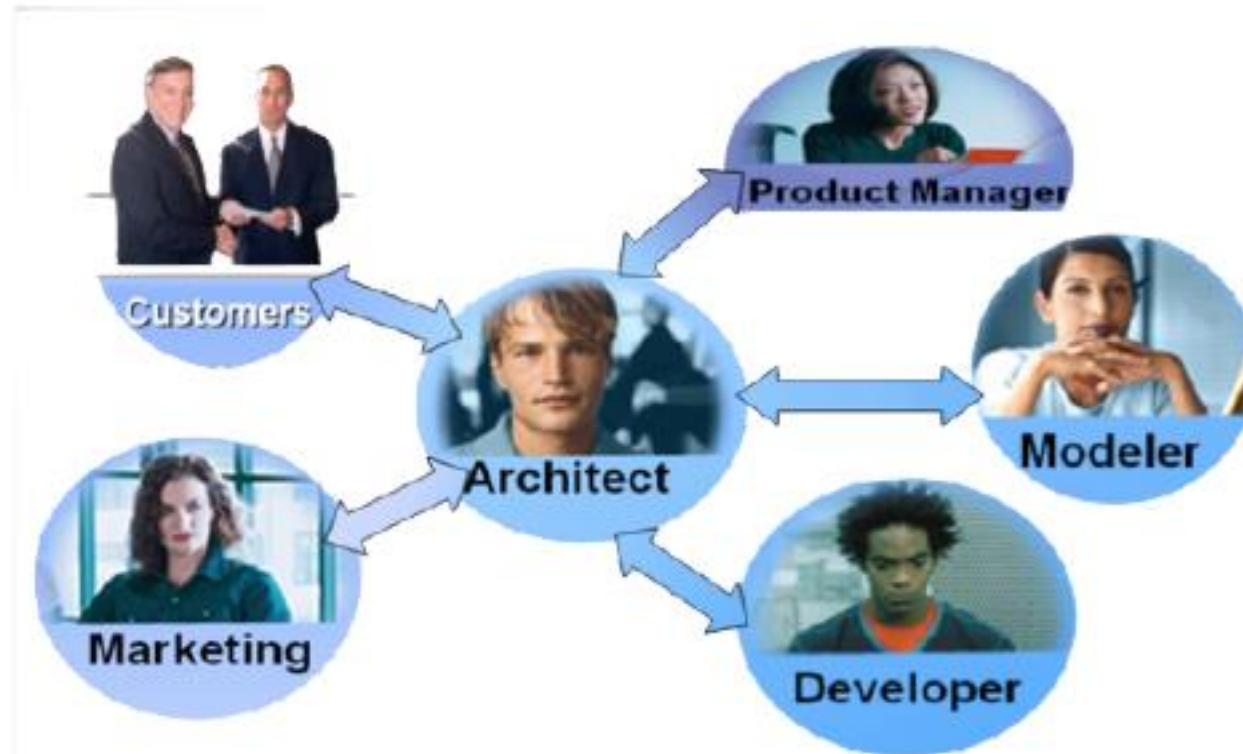


# When Sw. Architecture Start ?



# The Software Architect

*Is the one that takes strategic design decision*



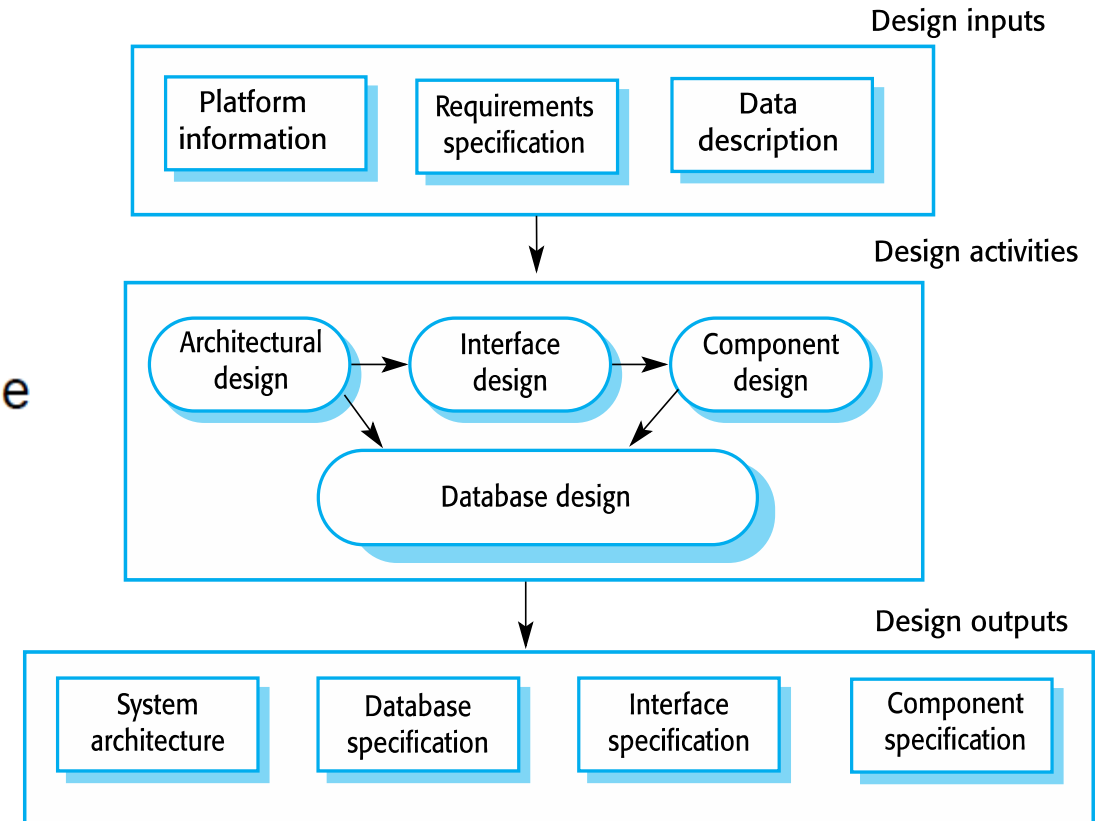
# What is Software Architecture?

“Software architecture is the **fundamental organization** of a system, embodied in its **components**, their **relationships** to one another and the environment, and the **principles** governing its design and evolution.”

- **IEEE Definition**

# Architectural Design

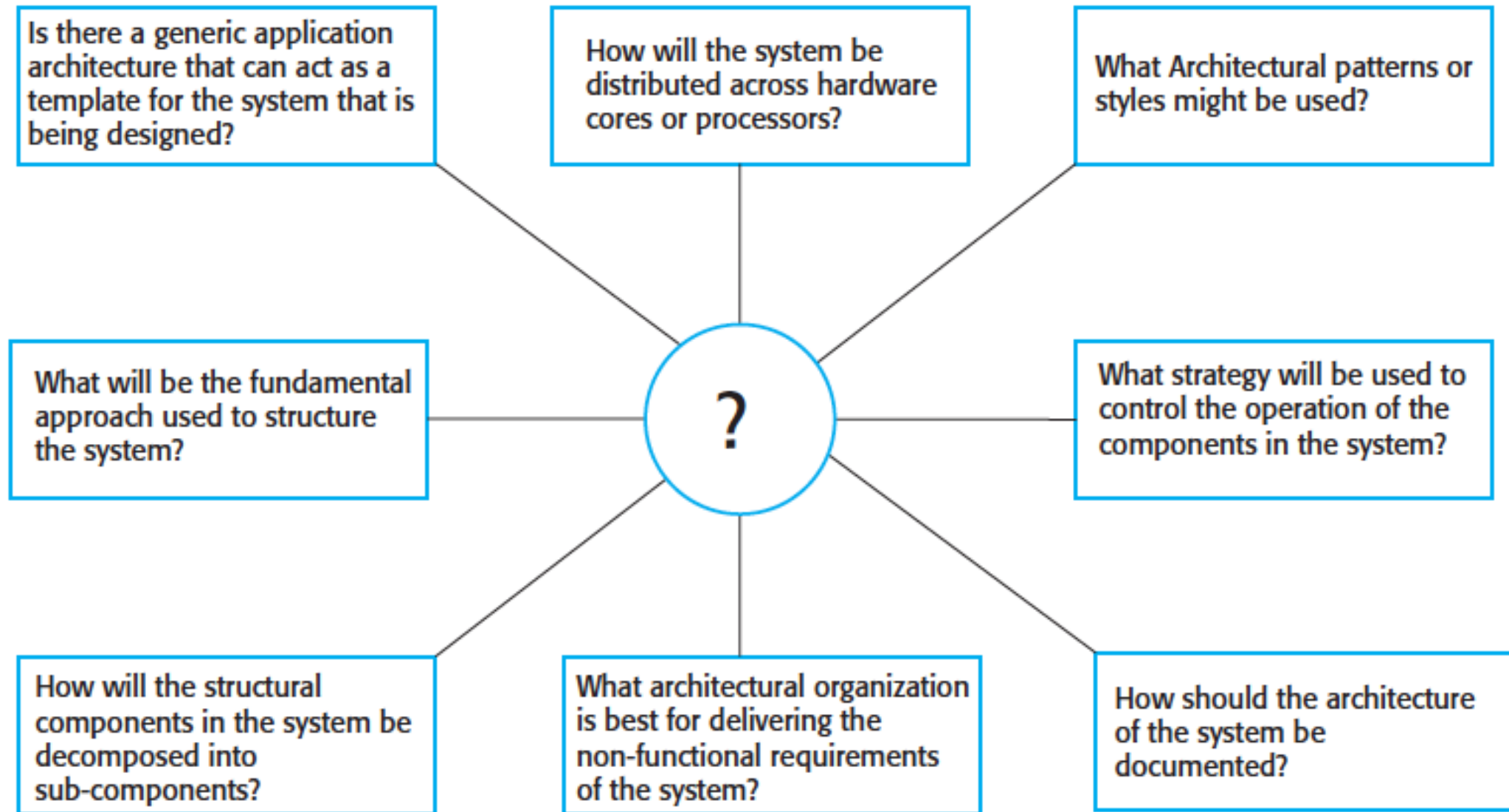
- First stage of design
- **Partitions** the requirements into self-contained subsystems.
  - Later, each subsystem will be decomposed into one or more classes.
- Plan how those subsystems cooperate and communicate.



# Architectural Design Decisions

- Design a system organization that will satisfy the functional and non functional requirements of a system
- Design architecture depends on the kind of systems, the requirements and the experience of the system architect
- A number of design decisions have to be taken in order to come up with a proper system architecture

# Architectural Design Decisions



# Architectural Design Decisions

- As the non-functional characteristics are closely related to software architecture, the choice of architecture depends on the non-functional requirements
  1. **Performance** – localize the critical operations with small number of components instead of distributing it over the network
  2. **Security** – use a layered structure of architecture
  3. **Safety** – the safety- related operations must be co-located in a single component
  4. **Availability** – include redundant components, so that they can be used in cases of failure preventing the system from stopping its operations
  5. **Maintainability** – use fine grain, self contained components that can be readily changed without too many dependencies with other components

# Why Explicitly Plan Architecture?

- **Enable stakeholder communication**
  - High-level presentation of the system.
- **Enables system analysis**
  - Can look for problems before coding.
- **Enables large-scale reuse**
  - Planning subsystems as independent entities allows their reuse in other systems.
- **Bad architectural design means bad security**
  - Controlling access is the first line of defense.

# How We Partition a System

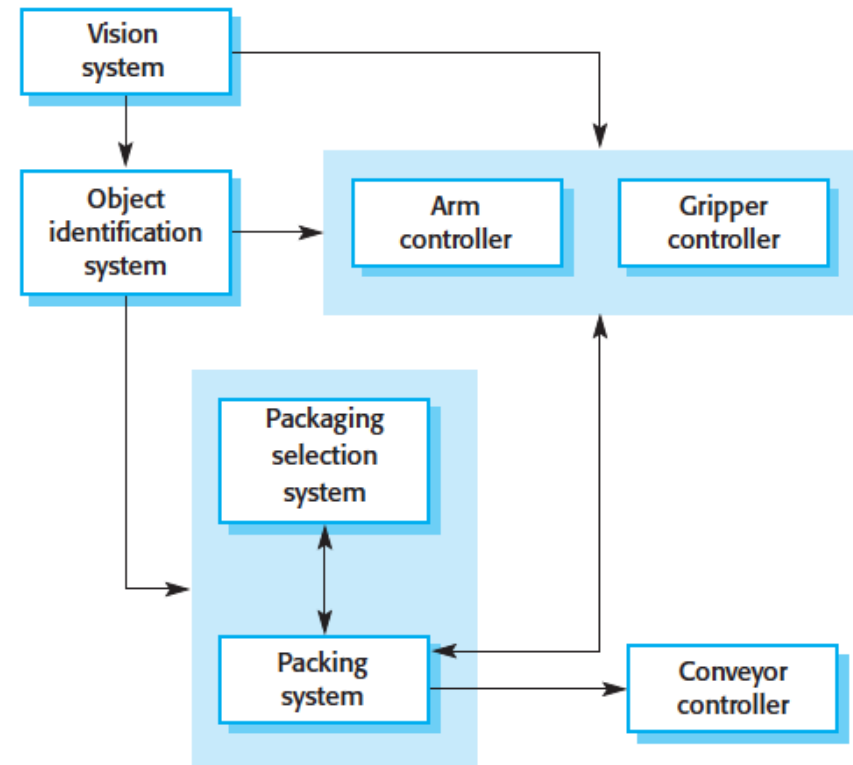
- System Structuring
  - The system is decomposed into several subsystems and connections between those subsystems are identified.
- Control Modeling
  - A model of the control relationship between different parts of the system is established.
- Modular Decomposition
  - The subsystems are decomposed into modules to structure the implementation.

# System Structuring and Architectural views

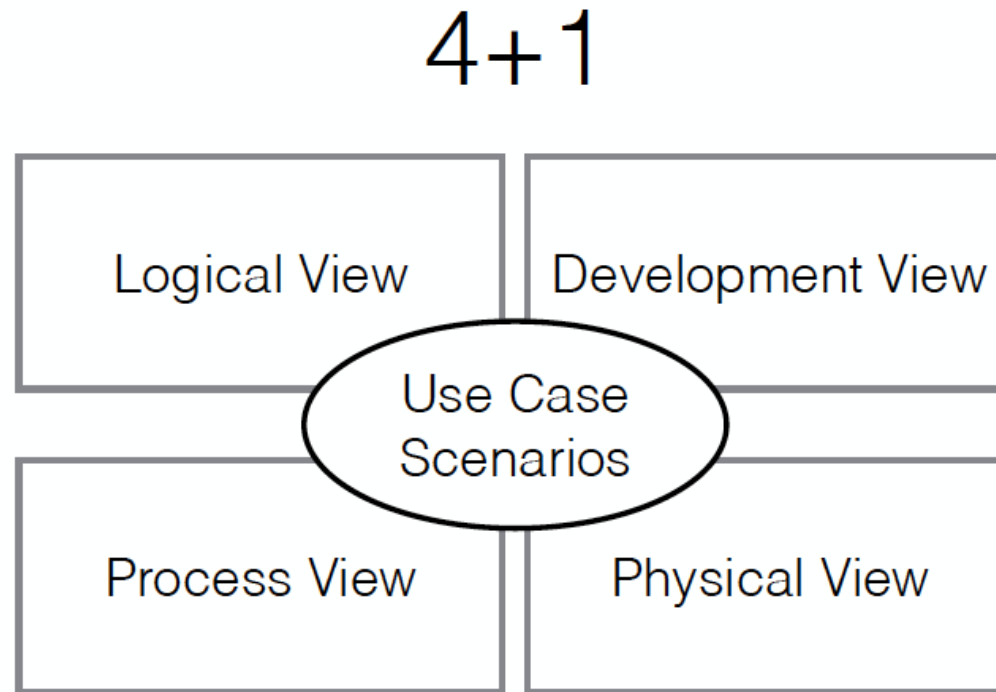
# System Structuring

- How we decompose the system into interacting subsystems.
- Can be visualized as block diagrams presenting an overview of the system structure.

## PACKING ROBOT SYSTEM



# Structuring / Architectural Views

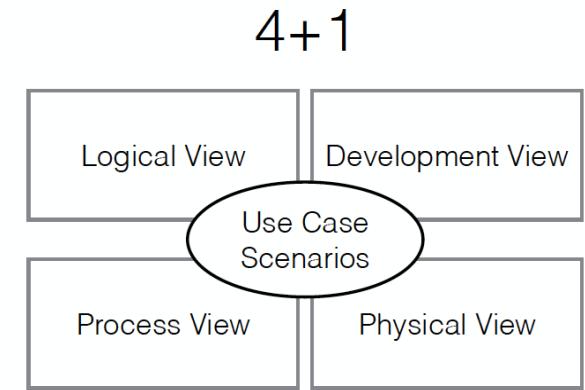


*Philippe Kruchten*

# Structuring Views

## Use Case Scenarios

- Unify and link the elements of the other 4 views
- Scenarios help to ensure that the architectural model is complete with respect to requirements
- The architecture can be broken down according to the scenarios and illustrated using the other 4 views

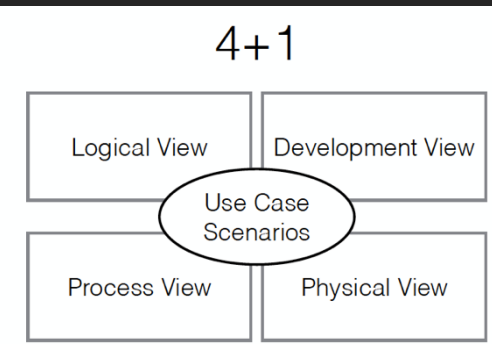


*Philippe Kruchten*

# Structuring Views - Example

## Music Player Scenarios

- Browse for new songs
- Pay to hear the entire song
- Download the purchased song on the phone
- Play the song

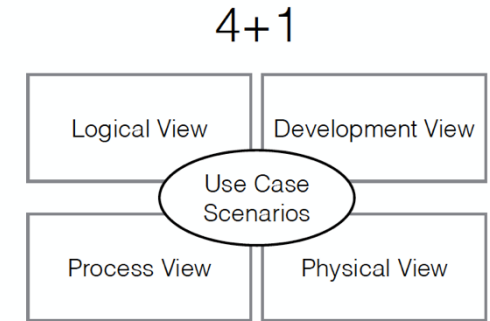


*Philippe Kruchten*

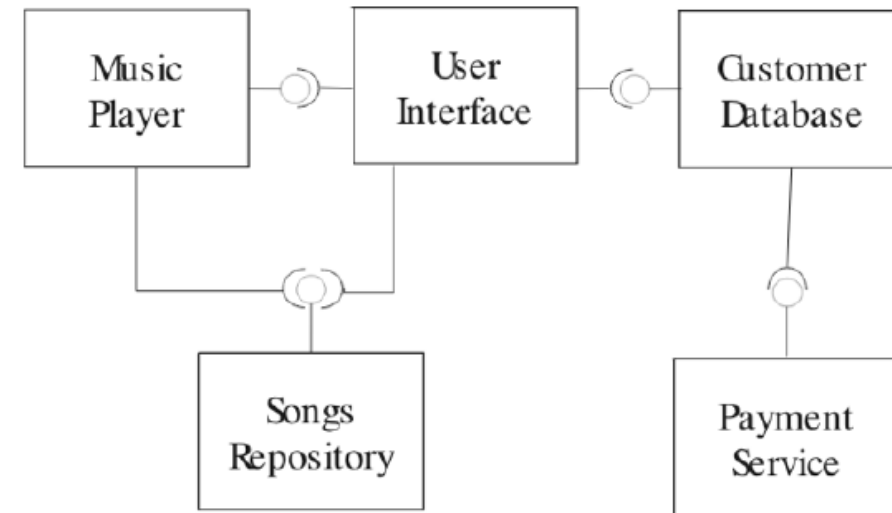
# Structuring Views

## Logical View

- Decompose the system structure into software components and connectors
- Map functionality (use cases) onto the components
  - **Concern:** Functionality
  - **Target Audience:** Developers and Users



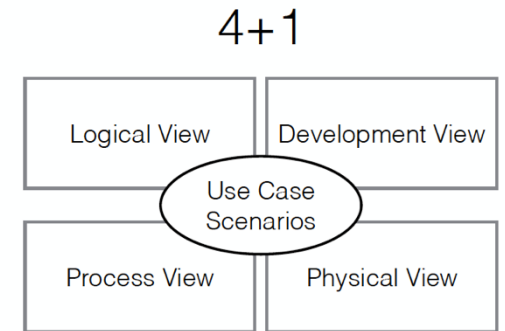
*Philippe Kruchten*



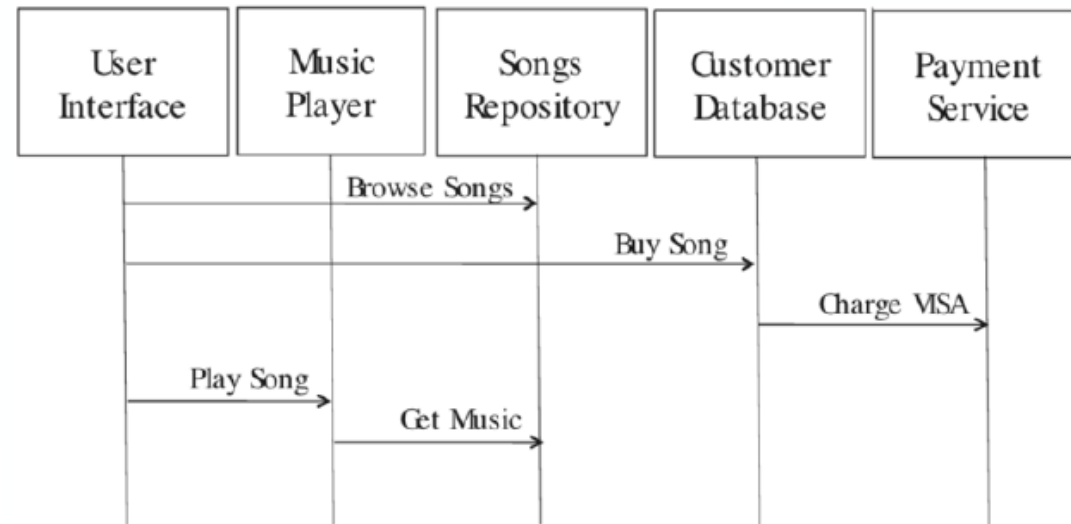
# Structuring Views

## Process View

- Model the dynamic aspects of the architecture and the behavior its parts
  - *active components*
  - *concurrent threads*
- Describe how processes/threads communicate
  - *RPC*
  - *Message bus*
- **Concern:** Functionality, Performance
- **Target Audience:** Developers



*Philippe Kruchten*

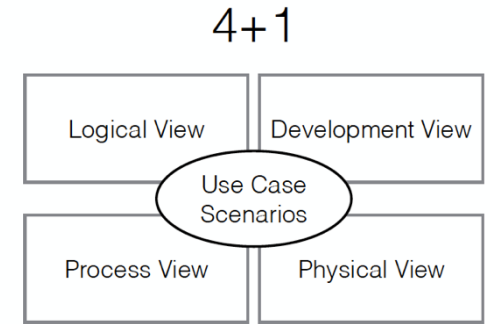
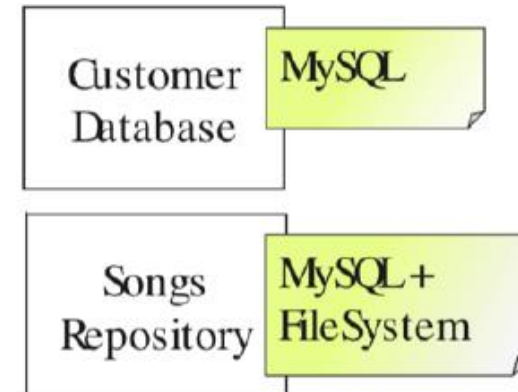
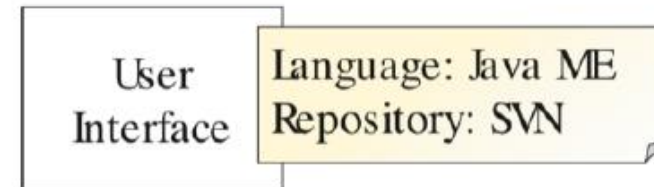


Use Cases: Browse, Pay and Play For Songs

# Structuring Views

## Development View

- Static organization of the software code artifacts
  - *packages*
  - *modules*
  - *binaries*
- Mapping between the elements in the logical view and the code artifacts
  - **Concern:** Reuse, Portability, Build
  - **Target Audience:** Developers



*Philippe Kruchten*

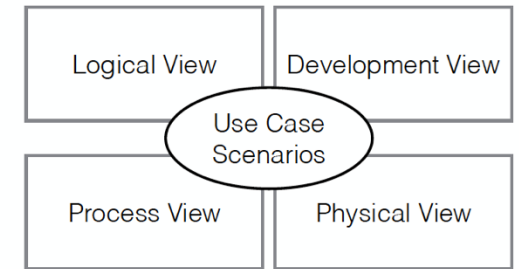


# Structuring Views

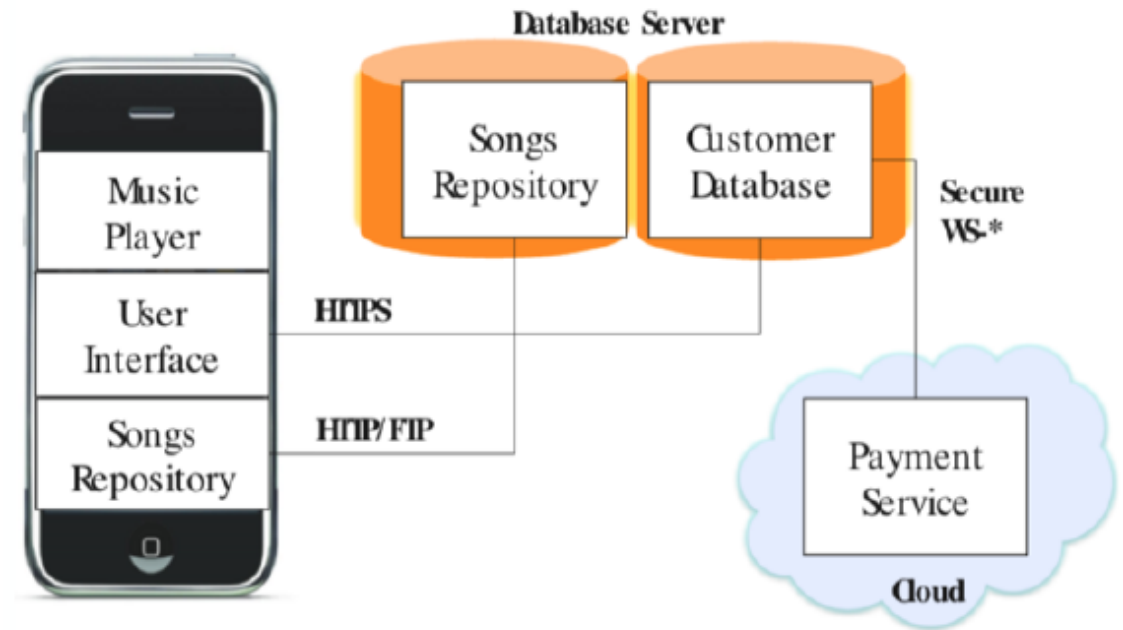
## Physical View

- Hardware environment where the software will be deployed
  - *hosts*
  - *networks*
  - *storage*
- Mapping between logical and physical entities
  - **Concern:** Quality attributes
  - **Target Audience:** Operations

4+1



*Philippe Kruchten*

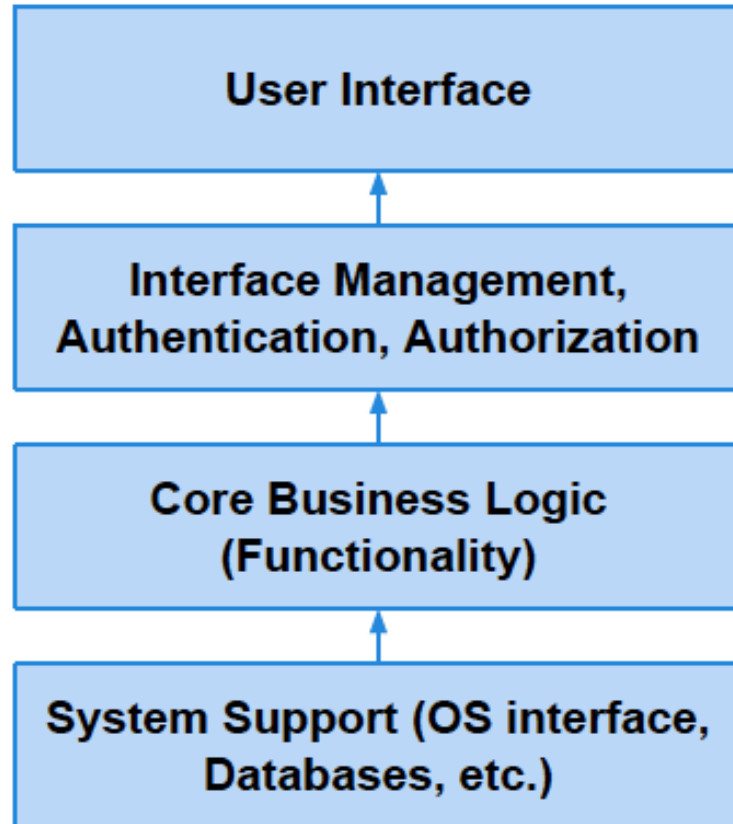


# Architectural Patterns

# Architectural Models

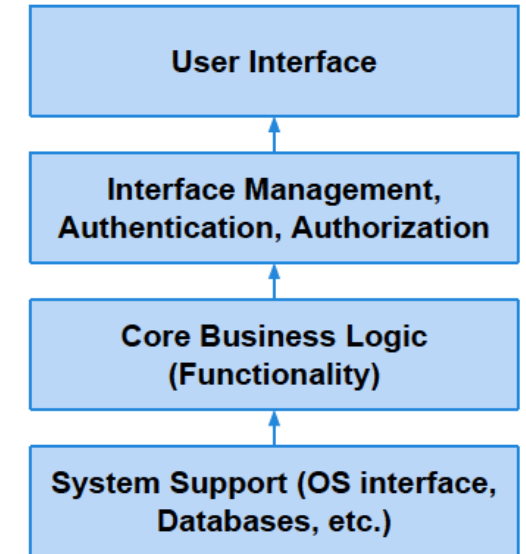
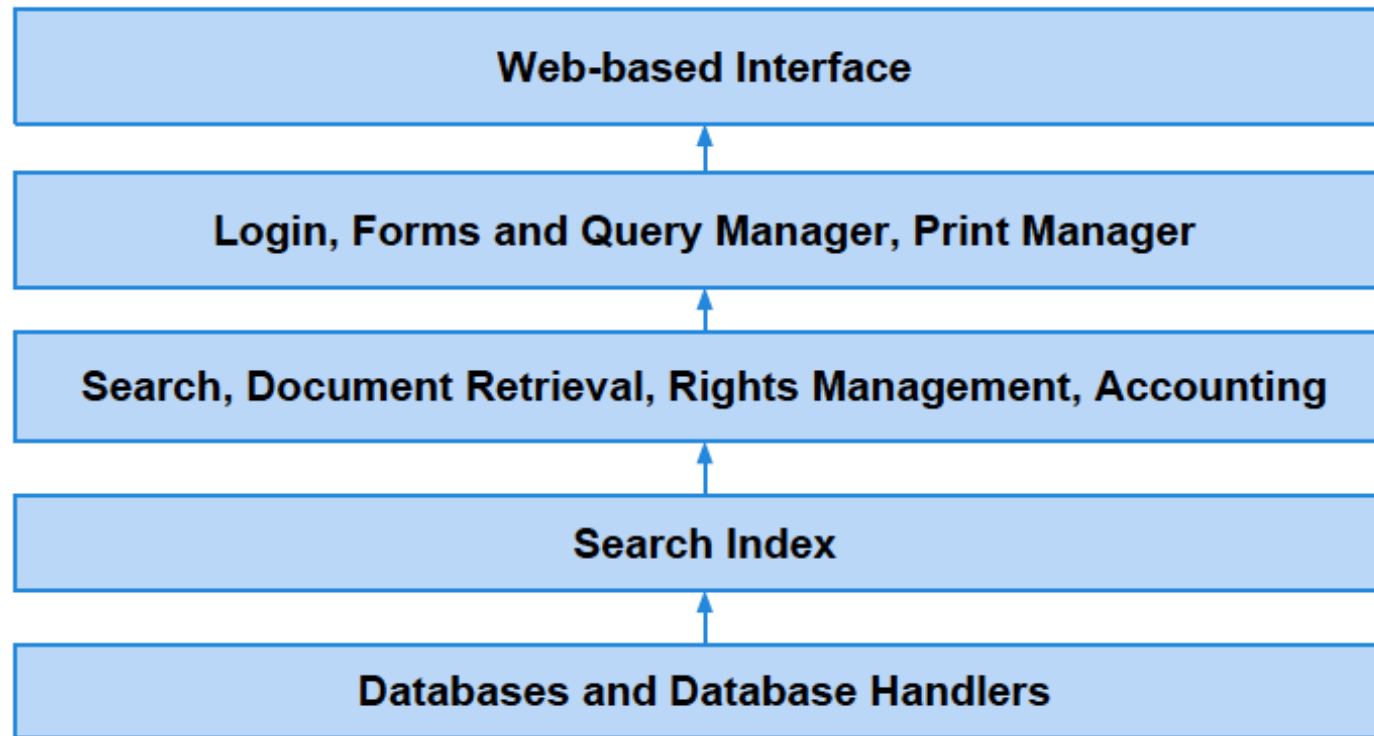
- Four common models: layered, shared repository, client/server, pipe & filter
- The model used affects the performance, robustness, availability, maintainability, etc. of the system.
- Complex systems might not follow a single model - mix and match.

# 1. Layered Model



- System functionality organized into layers, with each layer only dependent on the previous layer.
- Allows elements to change independently.
- Supports incremental development.

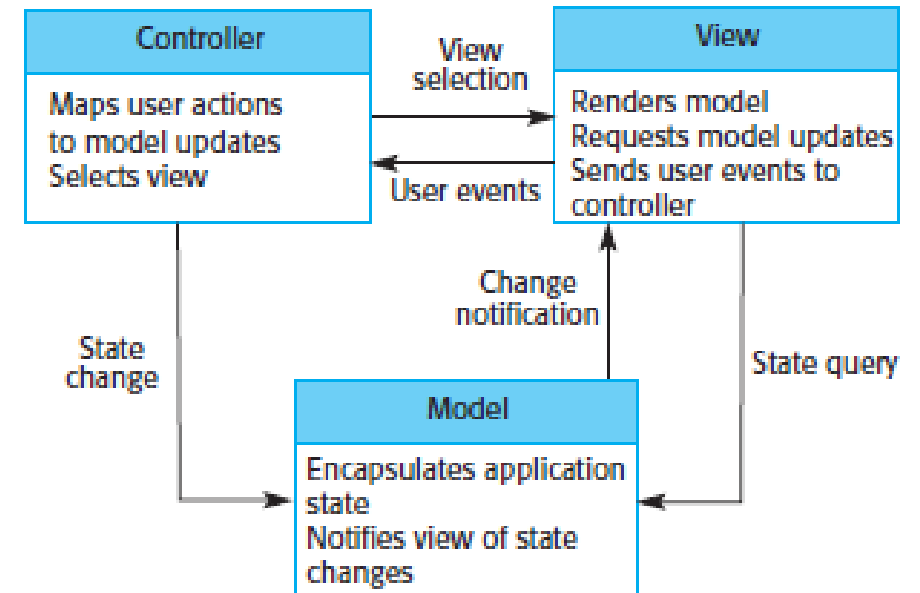
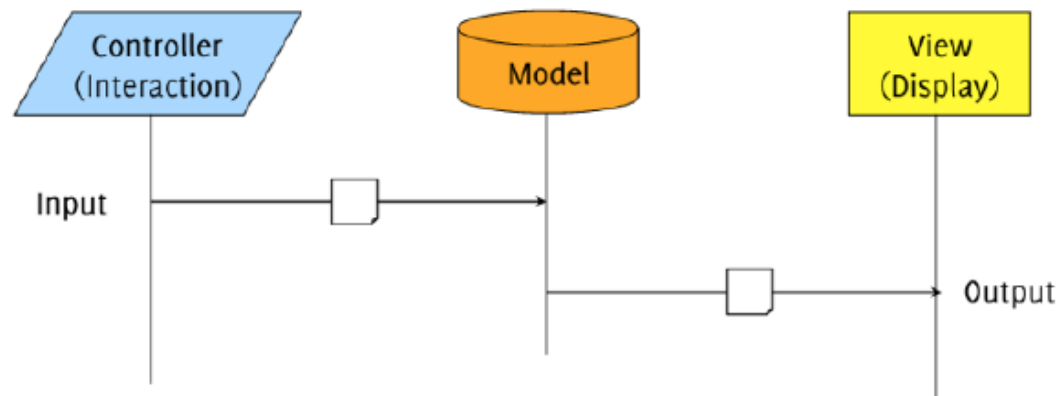
# Ex. Copyright Management Example



# 1. Layered Model

## Model-View-Controller

*separate content (model) from presentation (output) and interaction (input)*



# 1. Layered Model - Characteristics

## Advantages

- Allows replacement of entire layers as long as interface is maintained.
- When changes occur, only the adjacent layer is impacted.
- Redundant features (authentication) in each layer can enhance security and dependability.

## Disadvantages

- Clean separation between layers is often difficult.
- Performance can be a problem because of multiple layers of processing between call and return.

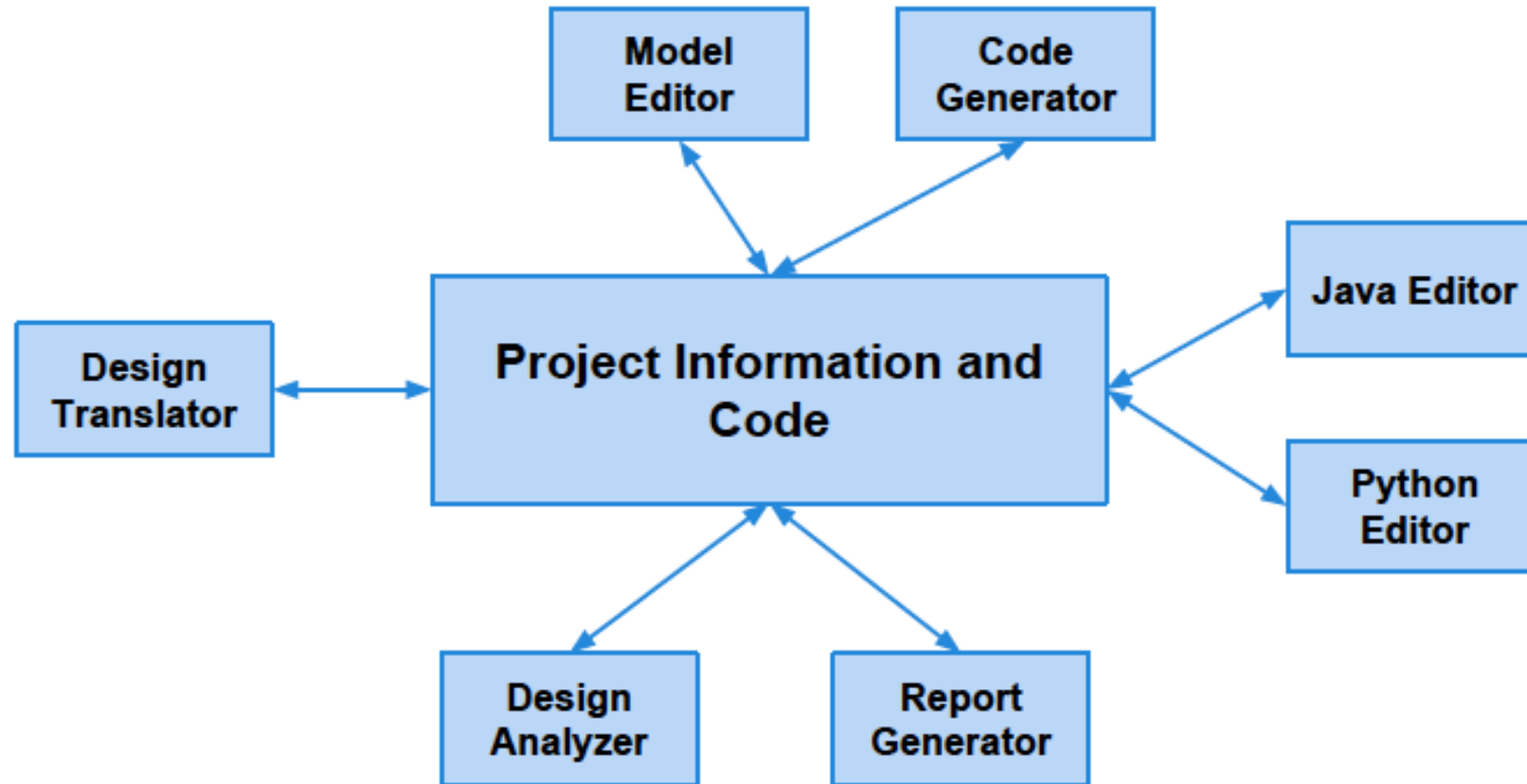
## 2. The Repository Model

Subsystems often exchange and work with the same data. This can be done in two ways:

- Each subsystem maintains its own database and passes data explicitly to other subsystems.
- **Shared data is held in a central repository and may be accessed by all subsystems.**

Repository model is structured around the latter.

# Ex. IDE Example



## 2. Repository Model Characteristics

### Advantages

- Efficient way to share large amounts of data.
- Components can be independent.
  - May be more secure.
- All data can be managed consistently (centralized backup, security, etc)

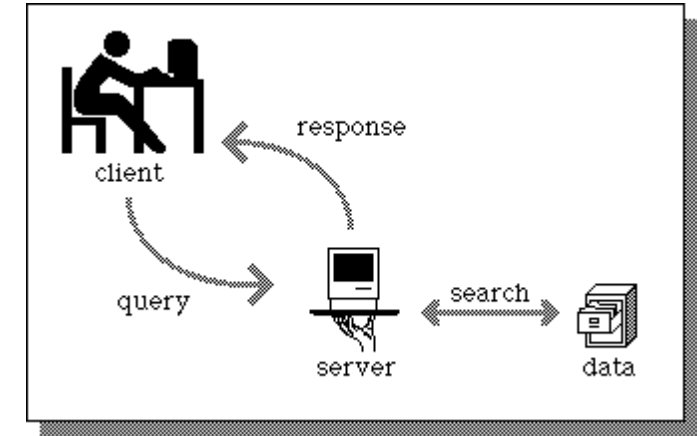
### Disadvantages

- Single point of failure.
- Subsystems must agree on a data model (inevitably a compromise).
- Data evolution is difficult and expensive.
- Communication may be inefficient.

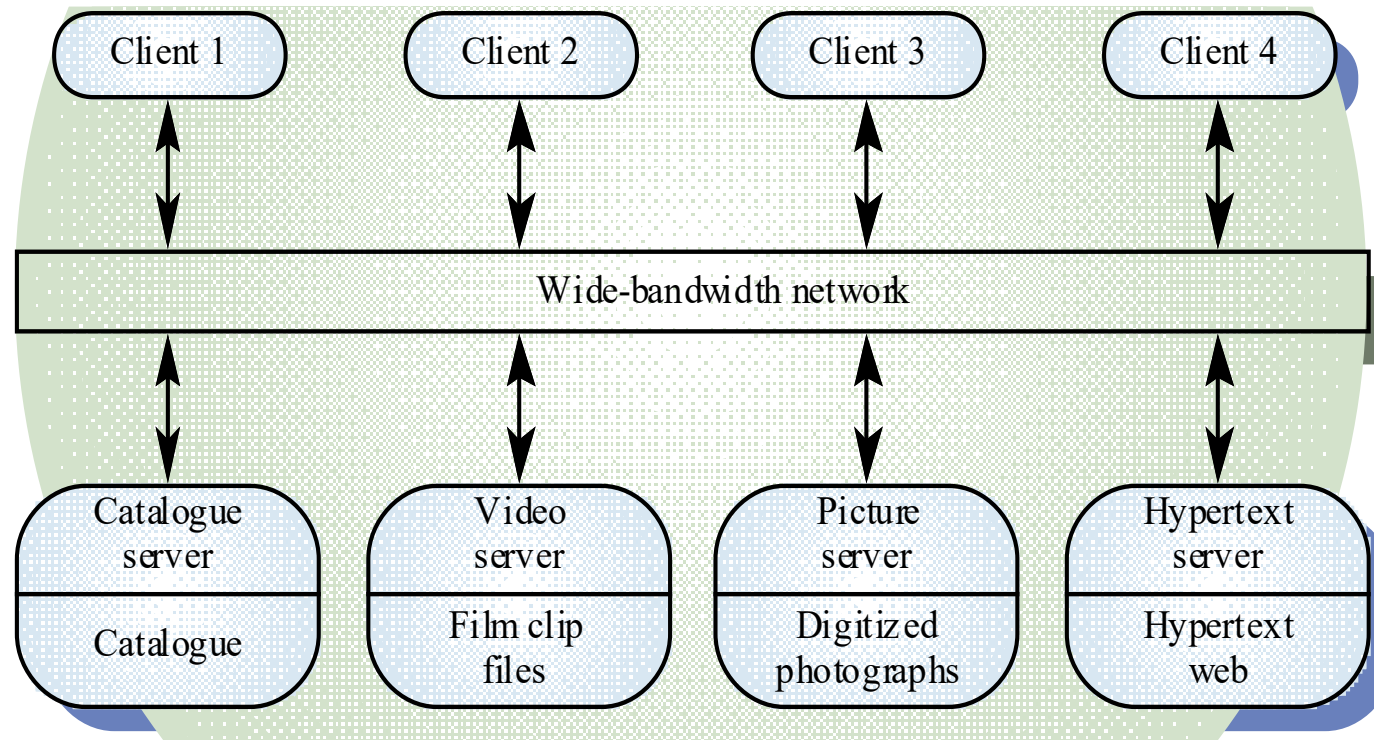
# 3. Client-Server Architecture

Functionality organized into services,  
distributed across a range of components:

- A set of servers that offer services.
  - Print server, file server, code compilation server, etc..
- Set of clients that call on these services.
  - Through locally-installed front-end.
- Network that allows clients to access these services.
  - Distributed systems connected across the internet.



# Ex: Film Library Example



# 3. Client-Server Model Characteristics

## Advantages

- Distributed architecture.
  - Failure in one server does not impact others.
- Makes effective use of networked systems and their CPUs. May allow cheaper hardware.
- Easy to add new servers or upgrade existing servers.

## Disadvantages

- Performance is unpredictable (depends on system and network).
- Each service is a point of failure.
- Data exchange may be inefficient (server -> client -> server).
- Management problems if servers owned by others.

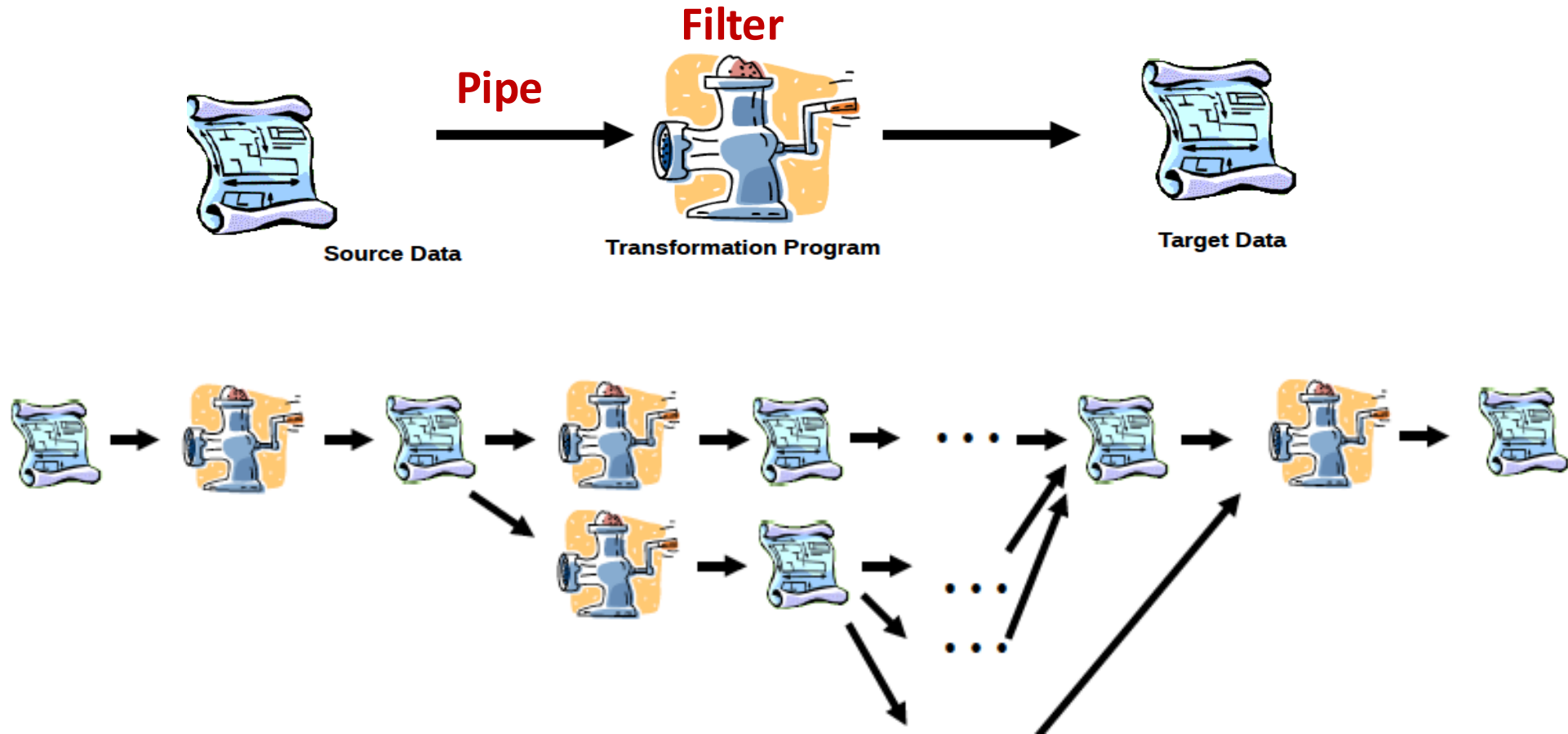
# 4. Pipe and Filter Architecture

Input is taken in by one component, processed, and the output serves as input to the next component.

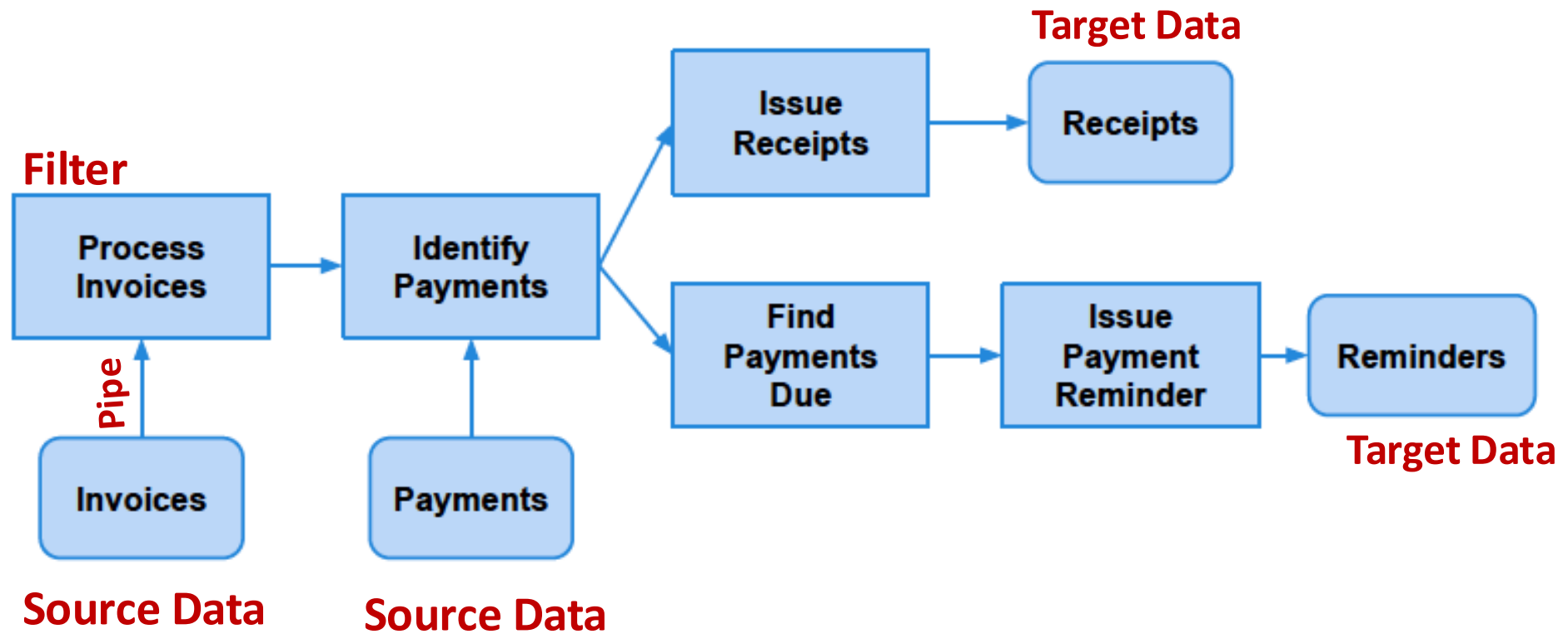
- Each processing step transforms data.
- Transformations may execute sequentially or in parallel.
- Data can be processed as items or batches.
- See Unix command line:

```
o cat file.txt | cut -d, -f 2 | sort -n |  
  uniq -c
```

# 4. Pipe and Filter Architecture



# Ex: Customer Invoicing Example



# 4. Pipe and Filter Characteristics

## Advantages

- Easy to understand communication between components.
- Supports subsystem reuse.
- Can add features by adding new subsystems to the sequence.

## Disadvantages

- Format for data communication must be agreed on. Each transformation needs to accept and output the right format.
- Increases system overhead.
- Can hurt reuse if code doesn't accept right data structure.

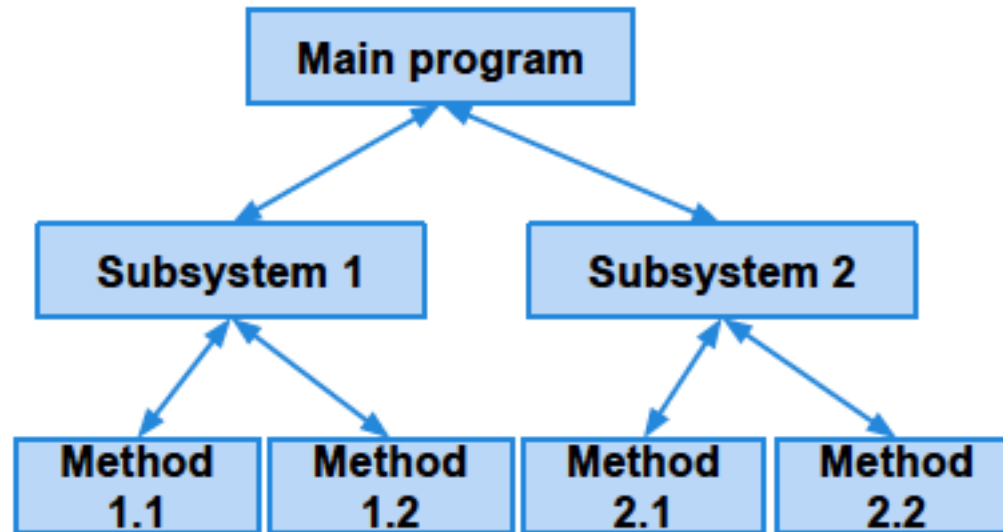
# Control Modeling

# Control Models (Not present in textbook)

- A model of the control relationships between the different parts of the system is established.
- During execution, how do the subsystems work together to respond to requests?
  - **Centralized Control:**
    - One subsystem has overall responsibility for control and stops/starts other subsystems.
  - **Event-Based Control:**
    - Each subsystem can respond to events generated by other subsystems or the environment.

# Centralized Control: Call-Return

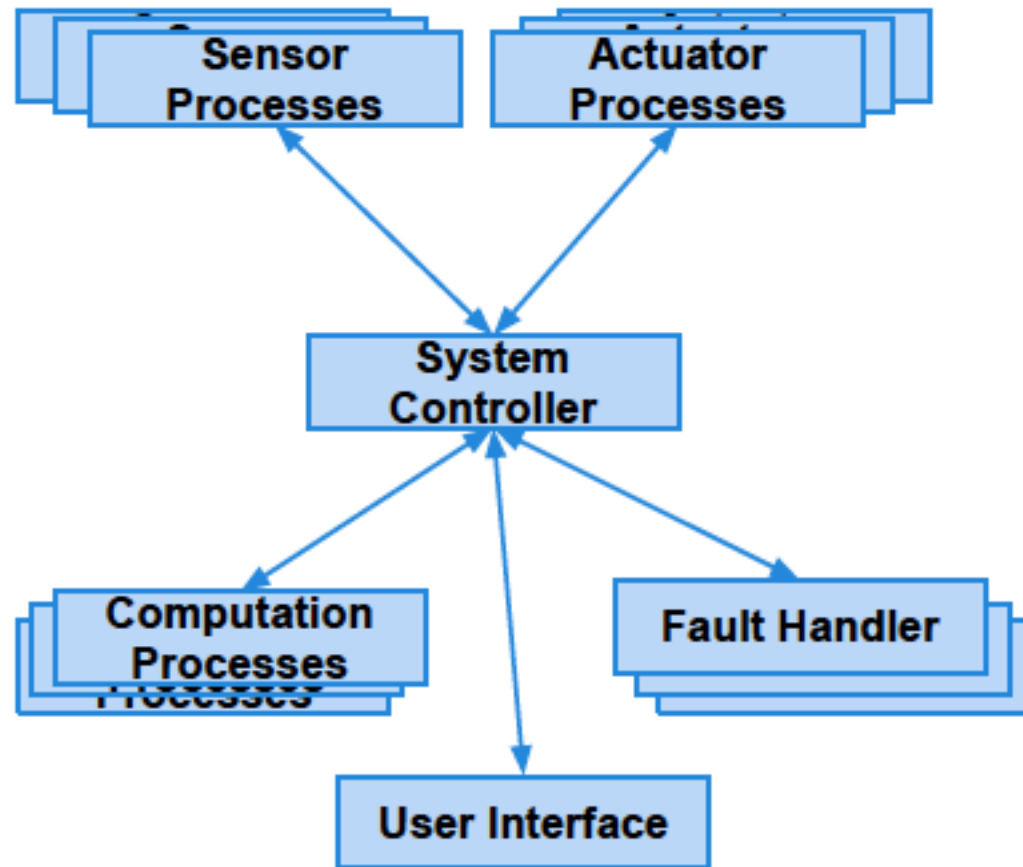
A central piece of code (Main) takes responsibility for managing the execution of other subsystems.



## Call-Return Model

- Applicable to sequential systems.
- Top-down model where control starts at the top of a subroutine and moves downwards.

# Centralized Control: Manager Model



## Manager Model

- Applicable to concurrent systems.
- One process controls the stopping, starting, and coordination of other system processes.

# Decentralized Control: Event-Driven Systems

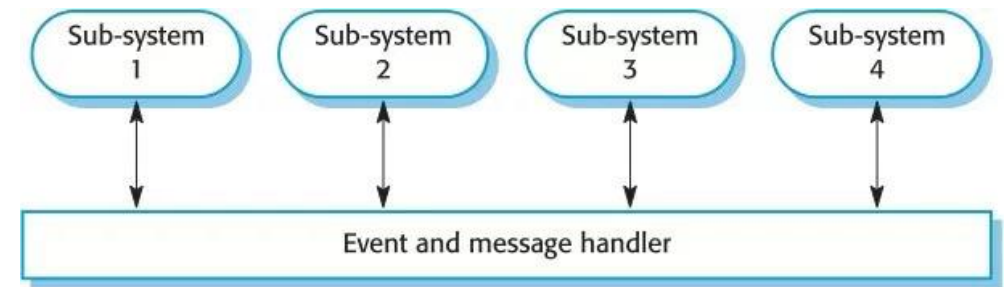
Control is driven by externally-generated events where the timing of the event is out of control of subsystems that process the event.

- **Broadcast Model**
  - An event is broadcast to all subsystems.
  - Any subsystem that needs to respond to the event does do.
- **Interrupt-Driven Model**
  - Events processed by interrupt handler and passed to proper component for processing.

# Broadcast Model

An event is broadcast to all subsystems, and any that can handle it respond.

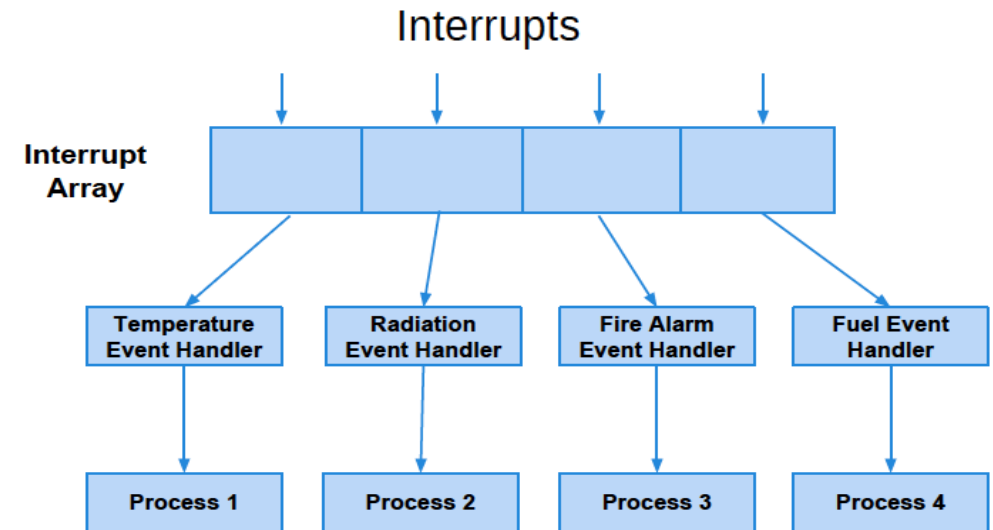
- Subsystems can register interest in specific events. When these occur, control is transferred to the registered subsystems.
- Effective for distributed systems. When one component fails, others can potentially respond.
  - However, subsystems don't know when or if an event will be handled.



# Interrupt-Driven Model

Events processed by interrupt handler and passed to proper component for processing.

- For each type of interrupt, define a handler that listens for the event and coordinates response.
- Each interrupt type associated with a memory location. Handlers watch that address.
- Used to ensure fast response to an event.
  - However, complex to program and hard to validate.



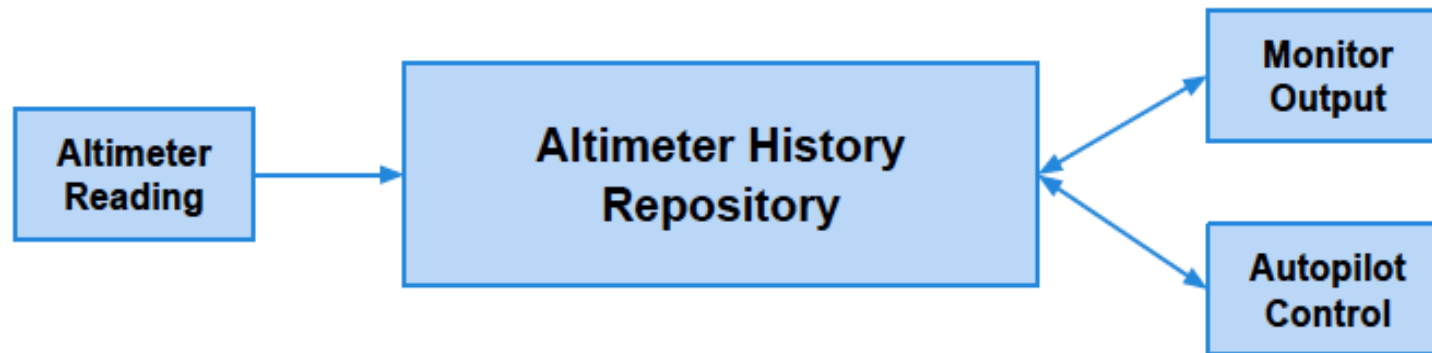
# Example: The ASW

You are designing control software for an aircraft. In such software, multiple behaviors are based on altitude. The software interfaces with one or more altimeters, makes autopilot decisions based on this information, and outputs status information to a monitor that is viewed by the pilot. If altitude drops below certain thresholds, the system will send warnings to that monitor and, if autopilot is engaged, will attempt to correct the plane's orientation.

- **Perform system structuring.** Try to use one or more of the models covered.
- **Perform control modeling.** How should control be routed?

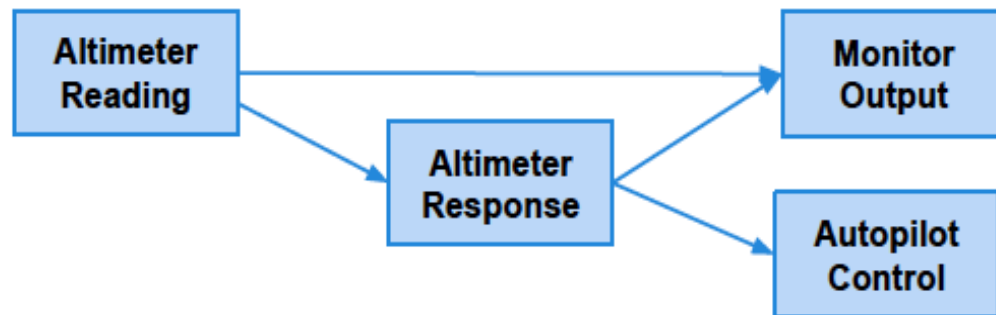
- **Perform system structuring. Try to use one or more of the models covered.**

## Option 1: Repository Model



- **Perform system structuring. Try to use one or more of the models covered.**

## Option 2: Pipe and Filter



- **Perform control modeling. How should events be handled?**

Depends on how you answered the previous question, but a natural option would be an Interrupt-Driven Model.

Handlers for new altimeter readings, for error flags triggered by altimeter processing code.

# Application Architectures

# Application Architectures Types

- **Data processing applications**

- Data driven applications that process data in batches without explicit user intervention during the processing.

- **Transaction processing applications**

- Data-centered applications that process user requests and update information in a system database.

- **Event processing systems**

- Applications where system actions depend on interpreting events from the system's environment.

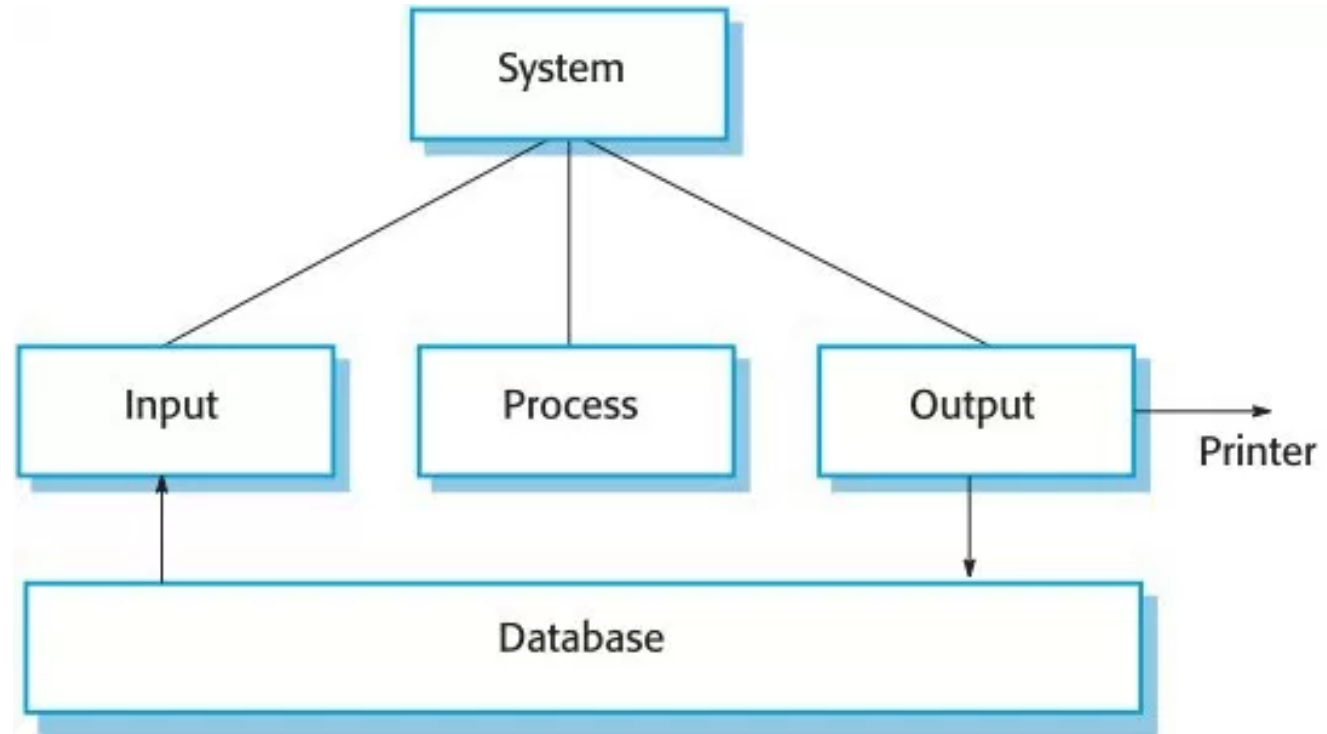
- **Language processing systems**

- Applications where the users' intentions are specified in a formal language that is processed and interpreted by the system.

# Application Type Examples

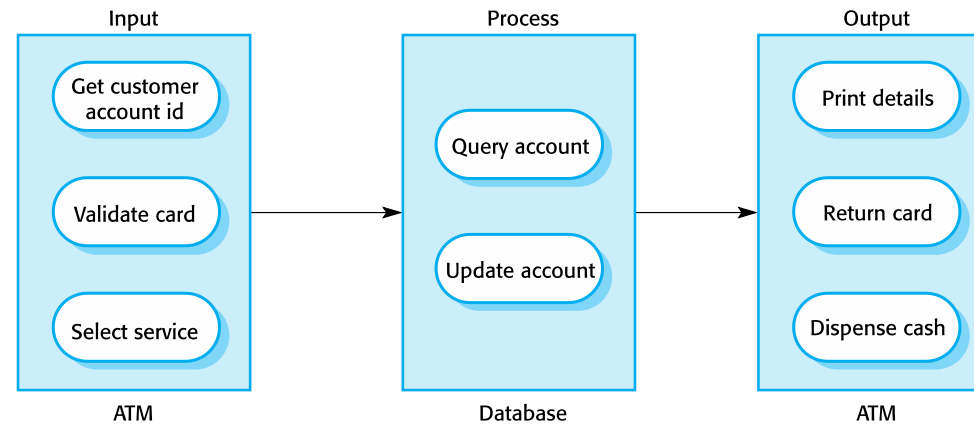
- Data processing systems
  - Billing systems;
  - Payroll systems.
- Transaction processing systems
  - E-commerce systems;
  - Reservation systems.
- Event processing systems
  - Word processors;
  - Real-time systems.
- Language processing systems
  - Compilers;
  - Command interpreters.

# Data Processing Systems

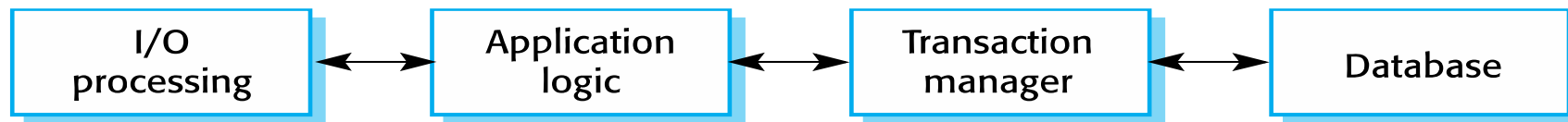


# Transaction Processing Systems

## The software Architecture of an ATM System

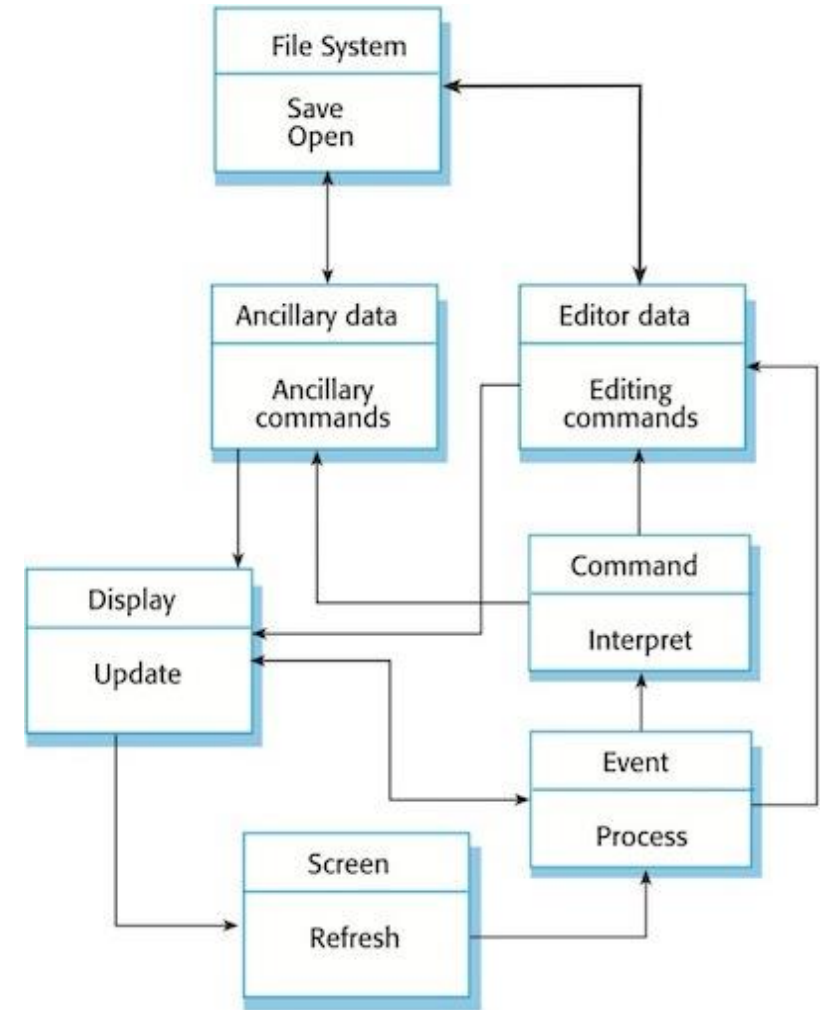


## Transaction Processing



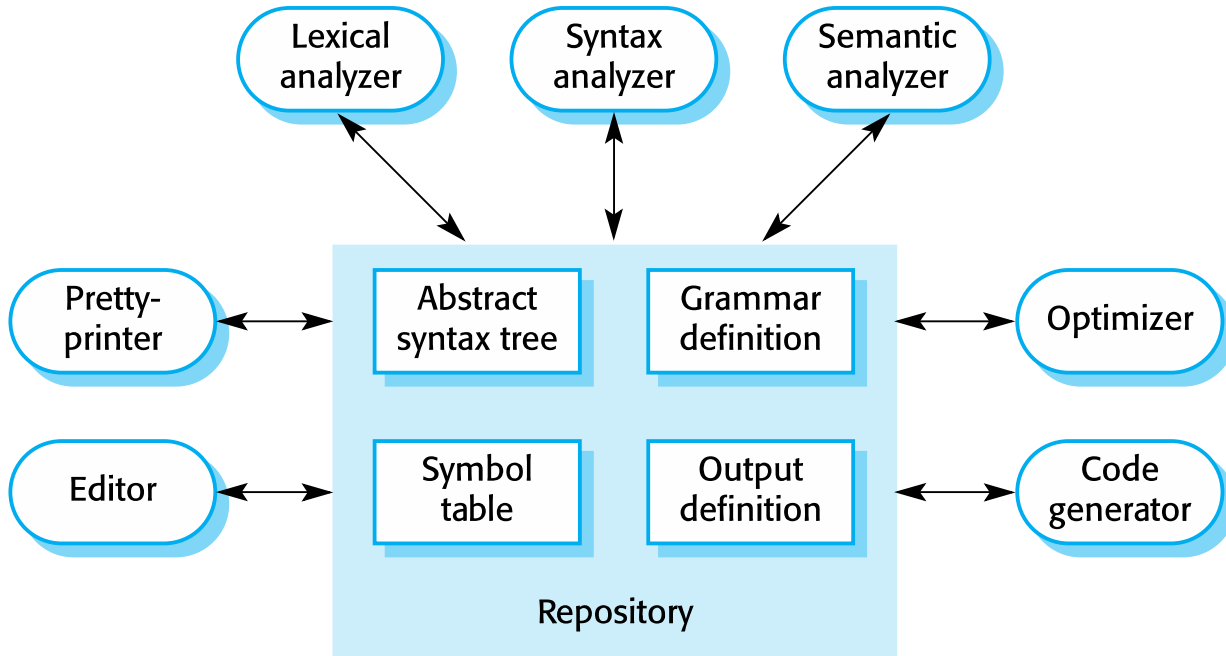
# Event processing systems

- These systems respond to events in the system's environment.
- Their key characteristic is that event timing is unpredictable, so the architecture has to be organized to handle this.
- Many common systems such as word processors, games, etc. are event processing systems.

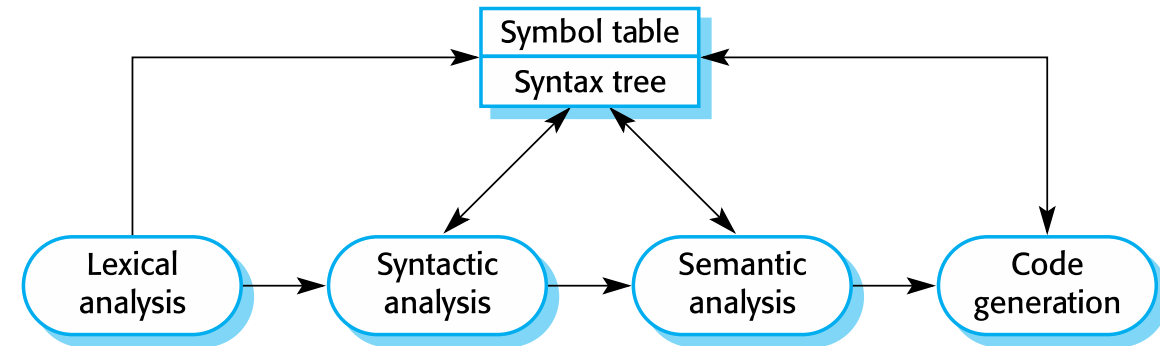


# Language Processing System - Example

## Repository Architecture



## Pipe and Filter Compiler Architecture



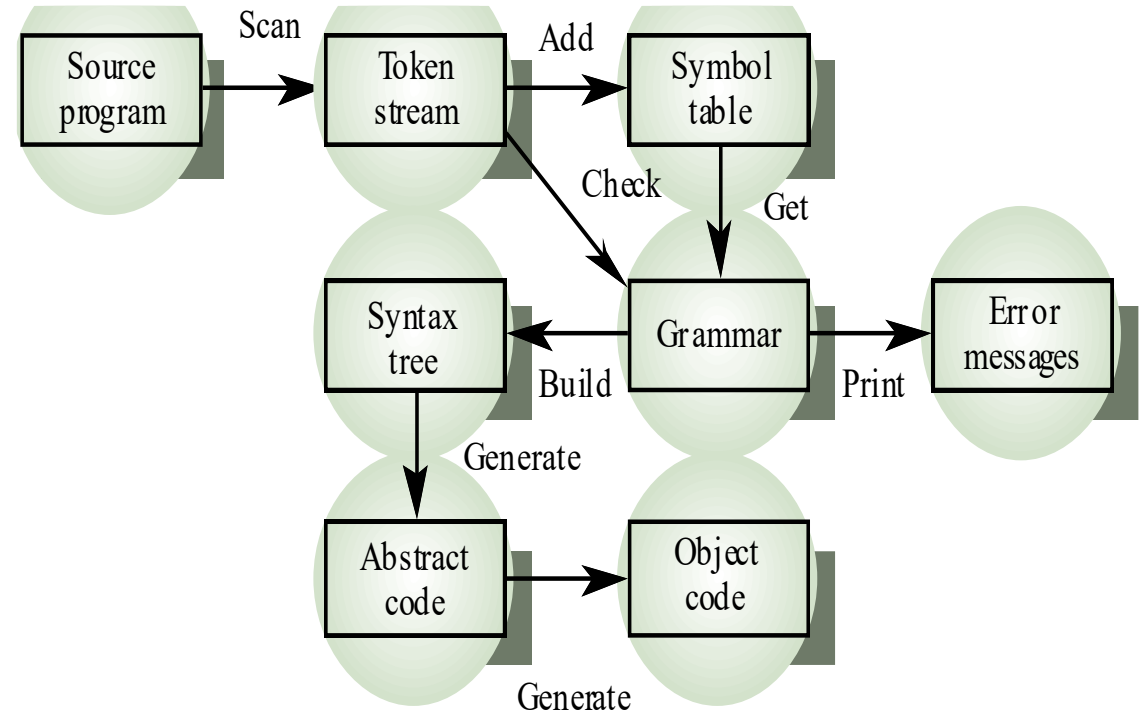
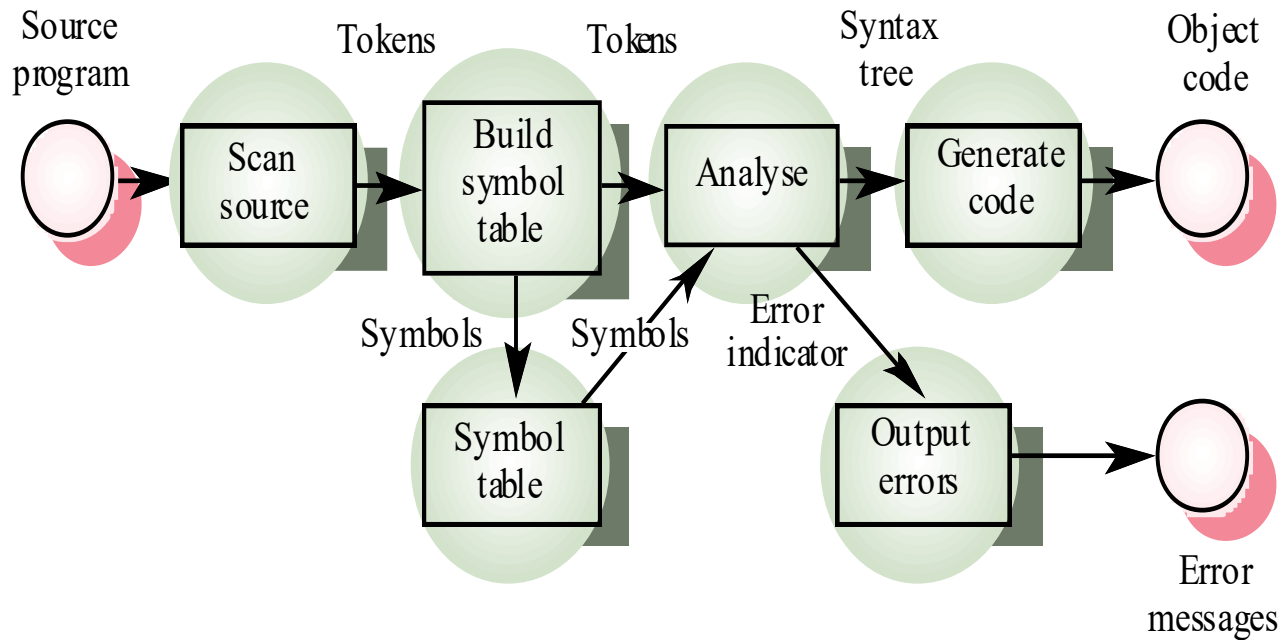
# Modular Decomposition

# Modular Decomposition (not present in textbook)

The rest of design - subsystems need to be decomposed into modules.

- How we get from a “subsystem” to classes and methods.
  - Two modular decomposition models covered
    - **An object model** where the system is decomposed into interacting objects
    - **A data-flow model** where the system is decomposed into functional modules which transform inputs to outputs.  
Also known as the pipeline model

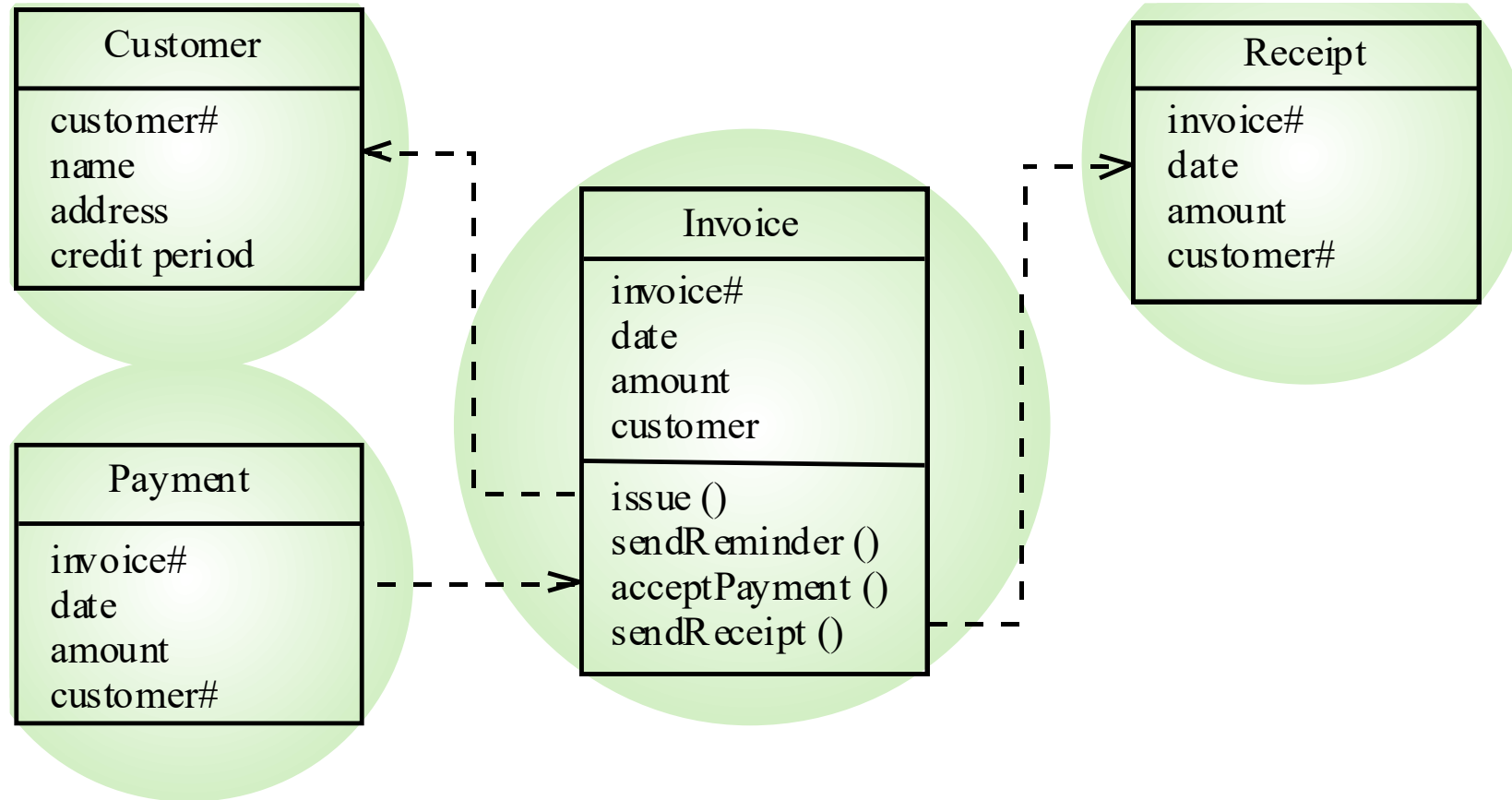
# OO vs Functional Views



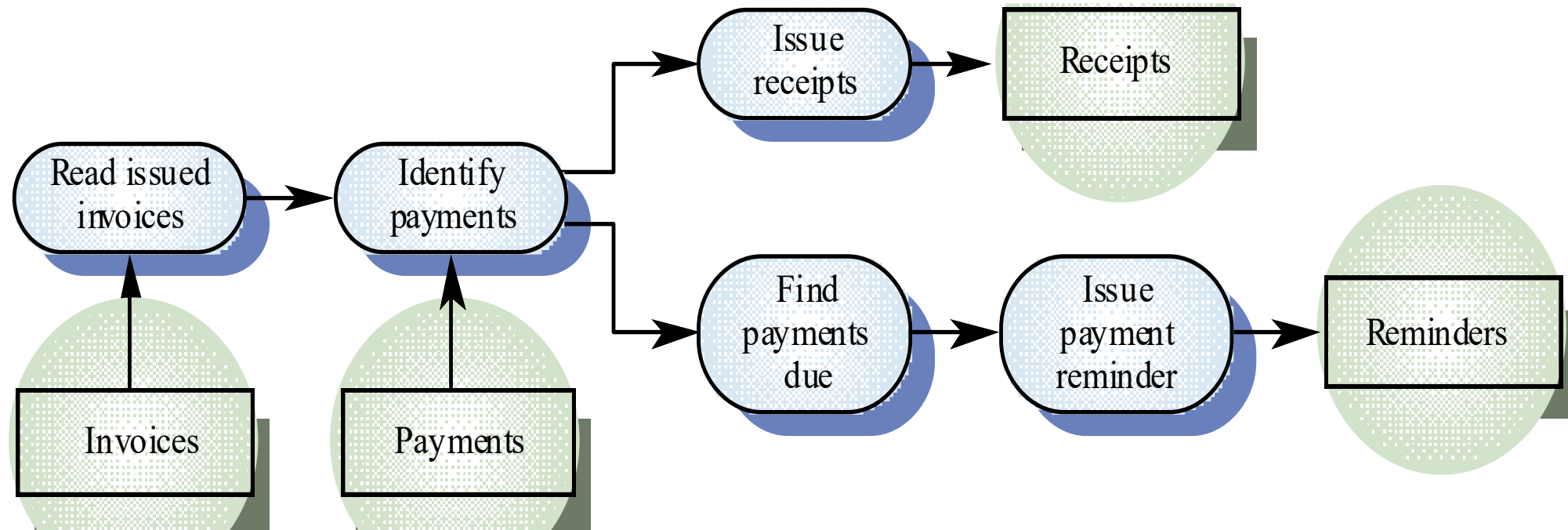
# Object Models

- Structure the system into a set of loosely coupled objects with well-defined interfaces
- Object-oriented decomposition is concerned with identifying
  - object classes,
  - their attributes,
  - operations
- When implemented,
  - objects are created from these classes
  - and some control model used to coordinate object operations

# Invoice Processing System OO model



# Invoice Processing System Functional View



# Key Points

- The software architect is responsible for deriving a system structure, a control model, and a modular decomposition.
- Architectural models can help organize a system.
  - Layered, repository, client-server, and pipe and filter models - also many others.
- Control models include centralized control and event-driven models.